Artificial Intelligence Algorithms for Electronic Component Recognition

Giovanni Cadau

Politecnico di Torino — MSc in Data Science and Engineering
in collaboration with Seica S.p.A., Strambino (TO), Italy

Advisor: Prof. Daniele Apiletti Company advisors: Paolo Chiartano, Ing. Francesco Grassino

Academic Year 2023/2024 — Paper-style summary of the Master's thesis

Abstract — In the Automatic Electronic Testing of Printed Circuit Boards (PCBs), a crucial and traditionally manual step is the recognition of specific electronic components such as resistors, capacitors, inductors, and integrated circuits. Manual recognition is time-consuming, error-prone, and requires a domain expert, while the heterogeneity of PCB designs and the variability of industrial acquisition conditions make automation challenging. This work develops a complete deep-learning pipeline for component recognition, from data collection and preprocessing to model deployment inside the production software of Flying Probe test systems. Eleven architectures are compared, ranging from Support Vector Machines with handcrafted features to transfer-learned Residual Networks; a ResNet152-based classifier reaches 95.0% test accuracy over 22 component classes. To make the model robust to environmental shift without retraining, an *Adaptive Domain Randomization* scheme is proposed in which the parameters of the augmentation distributions — including a full 7-dimensional multivariate normal over image properties — are optimized during training by the gradient-free CMA-ES algorithm running in a parallel branch of the network. On a deliberately domain-shifted dataset, accuracy improves from 82.5% to 99.9%. The classifier is extended to full-board object detection through a multi-scale sliding-window pipeline with image pyramids and Non-Maximum Suppression. Finally, thirty deployment-optimized variants combining data-reuse strategies, efficient layers, pruning, and quantization are benchmarked on the real industrial hardware: a binary quantized network reduces latency by 38%, memory by 66%, and energy by 40% while retaining 95.6% accuracy.

Keywords — Electronic component recognition, Printed Circuit Boards, Adaptive Domain Randomization, CMA-ES, model compression, industrial AI deployment.

1. Introduction

Electronic testing verifies that assembled Printed Circuit Boards behave as designed. In Flying Probe systems, mobile probes contact the board at programmed positions, and the correct configuration of a test depends on knowing *which* components are present and *where* they are located. In current practice this information is produced largely by hand: an operator, often a domain expert, inspects high-resolution board images and annotates each component. The process is slow, scales poorly with production volumes, and is subject to human error [18].

Automating this step with computer vision is attractive but non-trivial. PCB designs differ widely in layout, color, and density; components of the same functional class vary strongly in package, marking, and size; acquisition conditions (illumination, camera settings, compression) change across production sites; and manufacturing tolerances introduce further appearance variation. A useful industrial model must therefore not only be accurate on the data it was trained on, but remain reliable under distribution shift, and it must respect the latency, memory, and energy budgets of the machines that host it [17].

This work, carried out with Seica S.p.A., an Italian manufacturer of Automatic Test Equipment, addresses the problem end-to-end. The contributions are: (i) a reproducible data pipeline over Seica's proprietary image database covering 22 component classes; (ii) a systematic comparison of eleven classification architectures, from SVMs with handcrafted features to transfer-learned Residual Networks [8]; (iii) an *Adaptive Domain Randomization* (ADR) method that optimizes the augmentation distributions themselves with a gradient-free evolutionary strategy [2], yielding strong out-of-domain robustness; (iv) a sliding-window object-detection pipeline built on the classifier; and (v) an extensive deployment study of compression and acceleration techniques [6, 12] evaluated on the actual Flying Probe host hardware.

2. Data and Preprocessing

Training data consist of high-quality photographs of PCBs used in Seica systems, stored in the company database and annotated per component — partially by hand and partially through software assistance. Each annotation carries the component class together with geometric metadata (dimensions, shape, centroid, orientation). Figure 1 shows representative output of the final pipeline. The target taxonomy comprises 22 classes, including ceramic and tantalum capacitors, resistors and resistor networks, inductors, diodes, LEDs, integrated circuits, connectors, relays, fuses, switches, and the "not mounted" case.

The preprocessing pipeline loads images from the database, shuffles and splits them, resizes them to the network input resolution, normalizes pixel values, and one-hot encodes the labels. Throughput-oriented steps — caching, batching, and prefetching — overlap data preparation with training and keep the GPU saturated. The pipeline is implemented twice, on top of `tf.data` (TensorFlow 2.11) and of PyTorch datasets, with identical semantics.

Standard data augmentation operates on seven image properties: brightness, contrast, horizontal flip, vertical flip, hue,

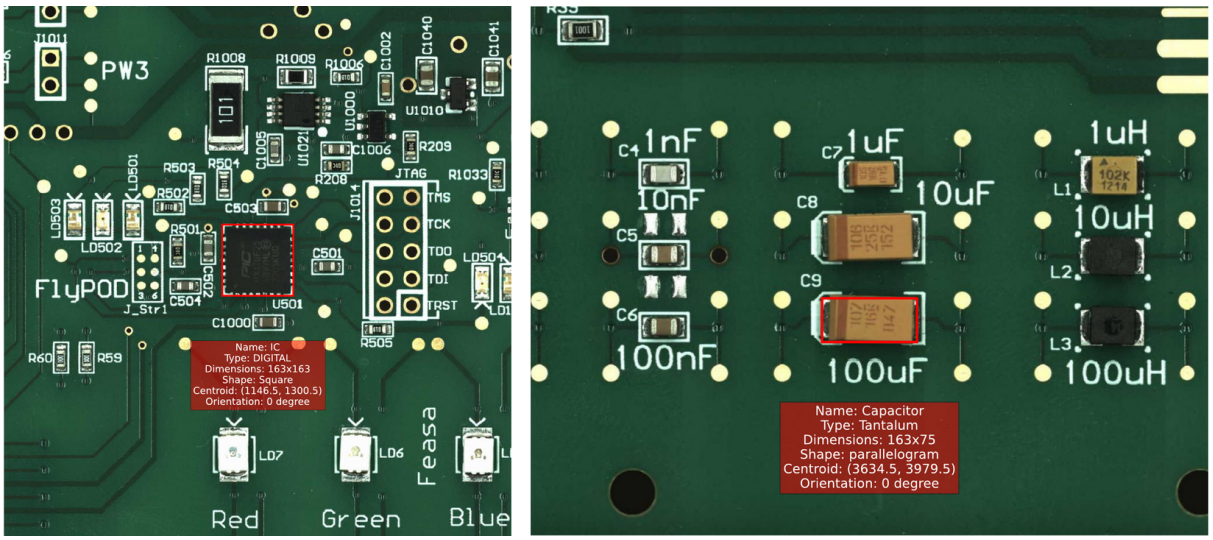

Figure 1: Output of the recognition pipeline on two PCB regions. For each detected component the model returns the bounding box, the class label and type, the dimensions in pixels, the shape, the centroid, and the orientation — the geometric quantities consumed by the test-programming software.

JPEG quality, and saturation. These seven properties also constitute the parameter set of the domain-randomization machinery of Section 4.

3. Model Architectures

Eleven models were designed and trained under identical conditions (Section 7). Models 1–3 are Support Vector Machines [3] operating on handcrafted representations: Histograms of Oriented Gradients with linear and RBF kernels, and a PCA-compressed representation with a polynomial kernel. Models 4–6 are convolutional networks of increasing depth (two to four convolutional blocks with max pooling and a dense head), trained from scratch on the raw 3D image tensor. Models 7–11 are residual networks [8]: a ResNet18 trained from scratch, and four ResNet152 variants pretrained on ImageNet [10] — with the backbone either fine-tuned or frozen, and with either a single 1000-unit dense layer or a deeper 1024–2048–1024 head before the softmax output.

4. Adaptive Domain Randomization

Domain Randomization (DR) [19] improves generalization by exposing the model, during training, to artificial variations of the input domain. At each batch, every parameter i in the set $\mathbf{P}$ of the seven image properties is randomized with probability p_i: a value is sampled from a parameter distribution $\mathscr{D}_i$ and an augmented image is generated accordingly. Four families of distributions are supported: uniform $\mathscr{U}(a_i, b_i)$, triangular $\mathscr{T}(a_i, m_i, b_i)$, univariate normal $\mathscr{N}(\mu_i, \sigma_i^2)$, and a single 7-dimensional multivariate normal

$$\mathscr{N}_7(\mu, \Sigma), \tag{1}$$

whose full covariance Σ captures correlations between augmentation parameters (e.g., between brightness and contrast).

The practical difficulty of DR is choosing the distribution coefficients $a_i, b_i, m_i, \mu_i, \sigma_i^2, \sigma_{ij}$: too little variation does not help, too much destroys the training signal. *Adaptive* Domain Randomization removes this manual tuning by treating

Algorithm 1 Adaptive Domain Randomization

Require: parameter set $\mathbf{P}$ to be randomized
Require: probability vector $\mathbf{p}$, $\mathbf{p}_i$ = probability that parameter i is randomized
Require: number of epochs E, batch size B
Require: distribution $\mathscr{D}_i(\mathbf{c}_i)$ per parameter, with coefficients $\mathbf{c}_i$ to be optimized

for each epoch $e = 1 \dots E$ **do**
  for each batch $b = 1 \dots B$ **do**
    for each parameter $i \in \mathbf{P}$ **do**
      if rand() $> \mathbf{p}_i$ **then**
        sample a value for parameter i from $\mathscr{D}_i(\mathbf{c}_i)$
        generate the augmented image for training
        forward propagation through the network
        evaluate loss $\mathscr{L}$
        optimize $\mathbf{c}_i$ of $\mathscr{D}_i(\mathbf{c}_i)$ from $\mathscr{L}$ *(gradient-free)*
      end if
    end for
  end for
end for

the coefficients as decision variables of an outer optimization problem. A parallel branch is attached to the network: after each forward pass on augmented data, the training loss $\mathscr{L}$ is fed to a gradient-free optimizer that updates the coefficients of $\mathscr{D}_i(\mathbf{c}_i)$, from which the next augmentations are sampled; Algorithm 1 summarizes the procedure. Because sampling and image generation are non-differentiable, gradient-free methods are required; all optimizers available in the Nevergrad library were tested, and CMA-ES [2] — which adapts the covariance of its own search distribution — consistently performed best, with Simulated Annealing the strongest alternative. Over training, the augmentation distributions visibly migrate and reshape (e.g., the contrast component first sharpens around a shifted mode, then widens again), automatically discovering the amount and correlation structure of nuisance variation that most helps generalization (Figure 2).

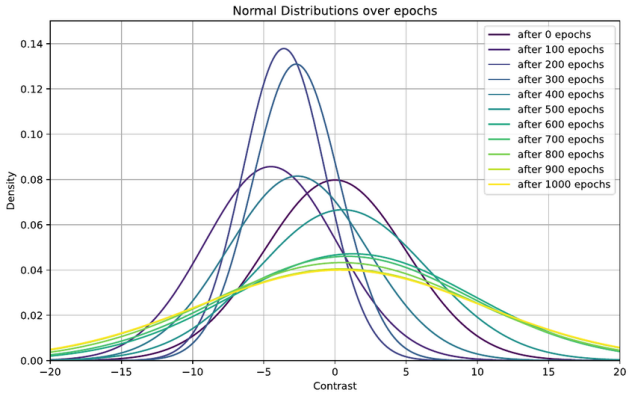

Figure 2: Contrast component of the 7-dimensional multivariate normal augmentation distribution, sampled every 100 epochs during CMA-ES optimization. The distribution first narrows around a shifted mode, then progressively widens: the optimizer discovers both the location and the spread of the nuisance variation that most improves generalization.

5. From Classification to Detection

The trained classifier supports two inference modes. In *image classification*, a single board region is presented and the network returns the class-likelihood vector. In *object detection*, an entire PCB image is scanned with a sliding window; each window is preprocessed and classified, and overlapping candidates are merged by Non-Maximum Suppression under an IoU criterion [14]. To detect components of different physical sizes without committing to a window size, the input image is expanded into an image pyramid [1]: it is repeatedly rescaled by a factor $s > 1$ down to a minimum size, and the sliding window is applied at every scale. The output is, for each detected component, its bounding box, class label, confidence score, and centroid — the geometric quantities the test-programming software consumes. Batch processing and process/thread parallelism reduce end-to-end inference time on full boards.

6. Deployment Optimization

Industrial deployment adds constraints beyond accuracy: inference latency, memory usage, model size, and energy consumption [17, 15]. Two families of techniques are explored. The first maximizes *data reuse* across the memory hierarchy: batching for fully connected layers; weights reuse, input-feature-map reuse, and batching for convolutional layers. The second exploits the *tolerance of deep networks to approximation*: efficient layers (spatially and depthwise separable convolutions [9, 5], grouped convolutions, width and resolution multipliers), pruning (magnitude-based, weight-saliency-based [11], block- and node-structured, with constant or gradually increasing schedules [7]), and quantization (integer formats from 16 down to 1 bit, minifloat, dynamic fixed point, base-2 logarithmic [13], post-training and quantization-aware, up to fully binary networks [16], plus weight clustering [6, 4]).

Thirty optimized variants of the best model, each combining a data-reuse pattern with one or two approximation techniques, were generated and evaluated directly on the hardware that manages the Flying Probe system, each over 1,000 inferences on distinct images — simulating the number of windows produced by a full-board scan.

Table 1: Model comparison on the held-out test set (%).

ID	Architecture	Acc.	F1
1	SVM, HOG, linear kernel	21.61	21.61
2	SVM, HOG, RBF kernel	22.37	22.38
3	SVM, PCA, polynomial kernel	18.09	18.08
4	CNN, 2 conv. blocks	55.26	55.23
5	CNN, 3 conv. blocks	58.49	58.46
6	CNN, 4 conv. blocks	62.37	62.36
7	ResNet18 (from scratch)	87.13	87.09
8	ResNet152 + 1000d head	91.96	91.94
9	ResNet152 (frozen) + 1000d head	90.52	90.50
10	**ResNet152 + 1024–2048–1024**	**95.03**	**95.01**
11	ResNet152 (frozen) + deep head	94.96	94.94

Table 2: Robustness on a domain-shifted dataset (Model 10): accuracy (%) by training regime.

Training regime	Test	Train
Baseline (no randomization)	82.46	99.98
DR, $\mathscr{U}$	89.91	91.45
DR, $\mathscr{T}$	90.31	92.65
DR, univariate $\mathscr{N}$	89.25	92.84
DR, multivariate $\mathscr{N}$	87.88	89.13
ADR, $\mathscr{U}$ + CMA-ES	93.24	94.22
ADR, $\mathscr{T}$ + CMA-ES	91.39	93.57
ADR, univariate $\mathscr{N}$ + CMA-ES	91.85	92.99
ADR, multivariate $\mathscr{N}$ + CMA-ES	**99.88**	**99.93**

7. Experimental Results

All eleven models of Section 3 were trained with 5-fold cross-validation; neural models used the Adam optimizer (learning rate 10^{-3}), categorical cross-entropy, and 1,000 epochs (loss and accuracy plateau at ∼420 epochs for the CNNs and full ResNets, ∼965 for the frozen variants). Table 1 reports test accuracy and F1.

SVMs on handcrafted features barely exceed chance level over 22 classes, confirming that the spatial complexity of PCB imagery defeats shallow pipelines. Plain CNNs recover much of the gap, and transfer-learned residual networks dominate: the best configuration, Model 10, attains 95.03% test accuracy (99.98% train) with precision, recall, and F1 all around 95%, and a log-scaled confusion matrix showing errors spread thinly across classes rather than concentrated in specific confusions.

Robustness. Model 10 was then re-evaluated on a *new* dataset whose images differ markedly from the training distribution (Table 2). Without randomization, accuracy drops to 82.46% despite near-perfect training accuracy — a classic domain gap. Fixed-coefficient DR recovers 5–8 points; adaptive optimization adds several more for every distribution family; and the combination of the multivariate normal with CMA-ES essentially closes the gap at 99.88%, indicating that the learned covariance across augmentation parameters is the key ingredient. Robustness of this kind translates directly into fewer retraining cycles when acquisition condi-

Table 3: Deployment on Flying Probe host hardware, 1,000 inferences: selected variants of Model 10.

Variant	Acc. %	Lat. s	Mem. MB	En. Wh
Reference (Model 10)	**99.98**	587	763	21.32
v1: separable conv. + magn. prun.	99.06	585	597	20.45
v8: grouped G=32 + 16-bit QAT	99.87	463	501	16.23
v20: 16-bit QAT, base-2 log	99.67	444	488	15.69
v21: grouped G=64 + sal. prun.	99.91	549	581	19.21
v22: 16-bit minifloat QAT (W)	99.87	462	502	16.21
v27: **binary NN, QAT, W+A**	95.62	**362**	**258**	**12.75**

tions change on the production floor.

Deployment. Table 3 shows representative points of the 30-variant grid. Reference accuracy here is measured in-domain. Quantization-aware 16-bit variants (integer with base-2 logarithmic encoding, or minifloat) retain $\geq$99.7% accuracy while cutting latency by $\sim$20–25% and memory by $\sim$35%. Grouped convolutions combined with weight-saliency pruning even nudge accuracy to 99.91% at reduced cost. At the aggressive end, fully binary networks trained with quantization-aware training reach 362 s per 1,000 inferences ($-$38%), 258 MB ($-$66%), and 12.75 Wh ($-$40%) at 95.62% accuracy — an attractive operating point whenever throughput dominates the requirement mix. The grid gives the production team a menu of accuracy–efficiency trade-offs rather than a single compromise.

8. Conclusions and Future Work

This work demonstrates that a carefully engineered deep-learning pipeline can automate electronic-component recognition on PCBs to industrial standards: a transfer-learned ResNet152 reaches 95% multi-class accuracy; Adaptive Domain Randomization with CMA-ES-optimized multivariate augmentation distributions makes it robust to substantial domain shift without retraining; a pyramid-based sliding-window pipeline turns the classifier into a full-board detector; and a systematic compression study identifies deployment variants meeting the latency, memory, and energy budgets of real test equipment. The models have been integrated into the company's production pipeline, reducing manual labor and inspection time.

Future directions include semantic segmentation for pixel-level delineation of densely packed or overlapping components, continued training on new PCB designs and acquisition conditions, and a deeper analysis of class-confusion patterns to guide targeted data collection and feature refinement.

References

[1] E. H. Adelson, C. H. Anderson, J. R. Bergen, P. J. Burt, and J. M. Ogden. Pyramid methods in image processing. *RCA Engineer*, 29(6):33–41, 1984.

[2] A. Auger and N. Hansen. Tutorial CMA-ES: evolution strategies and covariance matrix adaptation. In *Proc. GECCO Companion*, pages 827–848, 2012.

[3] O. Chapelle, P. Haffner, and V. N. Vapnik. Support vector machines for histogram-based image classification. *IEEE Trans. Neural Networks*, 10(5):1055–1064, 1999.

[4] Y. Choi, M. El-Khamy, and J. Lee. Towards the limit of network quantization. *arXiv:1612.01543*, 2016.

[5] J. Guo, Y. Li, W. Lin, Y. Chen, and J. Li. Network decoupling: from regular to depthwise separable convolutions. *arXiv:1808.05517*, 2018.

[6] S. Han, H. Mao, and W. J. Dally. Deep compression: compressing deep neural networks with pruning, trained quantization and Huffman coding. *arXiv:1510.00149*, 2015.

[7] S. Han, J. Pool, J. Tran, and W. Dally. Learning both weights and connections for efficient neural networks. In *NeurIPS*, 2015.

[8] K. He, X. Zhang, S. Ren, and J. Sun. Deep residual learning for image recognition. In *Proc. IEEE CVPR*, pages 770–778, 2016.

[9] A. G. Howard et al. MobileNets: efficient convolutional neural networks for mobile vision applications. *arXiv:1704.04861*, 2017.

[10] M. Huh, P. Agrawal, and A. A. Efros. What makes ImageNet good for transfer learning? *arXiv:1608.08614*, 2016.

[11] U. Kallakuri, E. Humes, and T. Mohsenin. Resource-aware saliency-guided differentiable pruning for deep neural networks. In *Proc. GLSVLSI*, pages 694–699, 2024.

[12] R. Krishnamoorthi. Quantizing deep convolutional networks for efficient inference: a whitepaper. *arXiv:1806.08342*, 2018.

[13] E. H. Lee, D. Miyashita, E. Chai, B. Murmann, and S. S. Wong. LogNet: energy-efficient neural networks using logarithmic computation. In *Proc. IEEE ICASSP*, pages 5900–5904, 2017.

[14] T.-Y. Lin et al. Microsoft COCO: common objects in context. In *Proc. ECCV*, pages 740–755, 2014.

[15] B. Moons, B. De Brabandere, L. Van Gool, and M. Verhelst. Energy-efficient ConvNets through approximate computing. In *Proc. IEEE WACV*, 2016.

[16] M. Rastegari, V. Ordonez, J. Redmon, and A. Farhadi. XNOR-Net: ImageNet classification using binary convolutional neural networks. In *Proc. ECCV*, pages 525–542, 2016.

[17] V. Sze, Y.-H. Chen, T.-J. Yang, and J. S. Emer. Efficient processing of deep neural networks: a tutorial and survey. *Proc. IEEE*, 105(12):2295–2329, 2017.

[18] E. M. Taha, E. Emary, and K. Moustafa. Automatic optical inspection for PCB manufacturing: a survey. *Int. J. Sci. Eng. Research*, 5(7):1095–1102, 2014.

[19] J. Tremblay et al. Training deep networks with synthetic data: bridging the reality gap by domain randomization. In *Proc. IEEE CVPR Workshops*, pages 969–977, 2018.